Escuela Superior de Economía y Negocios
Ingeniería de Software y Negocios Digitales

Asignatura:	**DevOps / Métricas de Software**
Nombre de la actividad:	**Proyecto de cátedra integrador**
Porcentaje:	DevOps: 30% (15% nota grupal – 15% defensa individual) Métricas de Software: 20% (10% nota grupal – 10% defensa individual)
Evidencia:	Servicio corriendo en Azure + repositorio de github con todo el código y evidencias + slides con explicaciones sobre los principales elementos de su solución
Fecha de entrega y presentación	**Semana de evaluaciones finales**

1. Propósito y alcance

Cada equipo desarrolla desde cero un sistema que resuelve uno de cinco escenarios empresariales y lo opera en Azure sobre un clúster Kubernetes (K3s) que construye y mantiene con infraestructura como código. El trabajo integra construcción de software, decisiones de arquitectura, integración y entrega continuas, Terraform, Kubernetes y observabilidad.

La evaluación no favorece la cantidad de servicios contratados, la sofisticación visual ni la complejidad arquitectónica por sí mismas. Se valora que las decisiones respondan a requisitos verificables, que el sistema pueda reconstruirse y que el equipo explique su comportamiento y diagnostique fallas con evidencia.

2. Organización y condiciones de trabajo

Las condiciones de trabajo son las mismas para todos los equipos; solo cambia el escenario asignado por el docente.

Aspecto	Condición
Equipos	Cuatro estudiantes, formados libremente
Nube pública	Microsoft Azure, suscripción Azure for Students, con un presupuesto máximo acumulado de US$100 por equipo
Disponibilidad	No se exige funcionamiento continuo; la infraestructura debe estar operativa durante demostraciones y evaluaciones programadas
Desarrollo	Libre elección de lenguajes, frameworks, arquitectura, persistencia y servicios auxiliares, dentro de las restricciones comunes
Inteligencia artificial	Permitida durante el desarrollo, con obligación de validar y defender el resultado; no permitida en la defensa individual ni en la comprobación individual

3. Requisitos técnicos comunes

Todos los equipos cumplen tres grupos de requisitos, cada uno con la evidencia mínima que se revisa en la defensa. No existe una cantidad obligatoria de microservicios, bases de datos, colas, frameworks, máquinas virtuales o herramientas de monitoreo; la implementación responde a los requisitos del escenario.

3.1. Aplicación y arquitectura

ID	Requisito	Evidencia mínima
COM-01	Implementar las funciones y reglas del escenario asignado.	Pruebas de aceptación ejecutables y resultados verificables.
COM-02	Definir y justificar los límites y responsabilidades de los componentes.	Diagrama de arquitectura real y descripción de decisiones.
COM-03	Persistir la información cuya conservación exija el escenario.	Demostración de persistencia ante reinicio de componentes.
COM-04	Manejar errores previsibles y validar entradas.	Pruebas positivas, negativas y de recuperación.
COM-05	Establecer controles de acceso apropiados cuando el escenario los requiera.	Pruebas de acceso autorizado y rechazado.
COM-06	Identificar los compromisos de consistencia, disponibilidad, costo y complejidad.	Registro breve de decisiones técnicas y alternativas descartadas.

3.2. DevOps

ID	Requisito	Evidencia mínima
DEV-01	Versionar aplicación, infraestructura y configuraciones no secretas en Git.	Repositorio e historial con pull requests revisados.
DEV-02	Distribuir los componentes ejecutables mediante imágenes de contenedor.	Dockerfiles o equivalentes, imágenes versionadas y trazables.
DEV-03	Ejecutar integración continua con GitHub Actions.	Pruebas y validaciones automáticas en PR; protección de rama que bloquea la integración cuando fallan.
DEV-04	Automatizar la construcción y publicación de imágenes.	Registro y vínculo entre commit, workflow e imagen desplegada.
DEV-05	Automatizar una entrega controlada, separando validación y producción de forma lógica.	Workflow, controles de aprobación y demostración de despliegue.
DEV-06	Definir mediante Terraform los recursos principales de Azure.	Código versionado, plan revisable, apply controlado.
DEV-07	Utilizar estado remoto para Terraform y proteger su acceso.	Configuración del backend y procedimiento de inicialización documentado.
DEV-08	Integrar la validación de Terraform en GitHub Actions.	Validación y plan ante cambios, sin aplicar automáticamente PR no aprobados.
DEV-09	Instalar y operar K3s en una o más máquinas virtuales de Azure.	Nodos y workloads verificables mediante Kubernetes.
DEV-10	Separar configuración y secretos de las imágenes; aplicar privilegio mínimo.	Mecanismos de configuración y evidencia de permisos.

ID	Requisito	Evidencia mínima
DEV-11	Configurar salud y recursos de workloads según sus características.	Probes apropiadas, requests y límites justificados.
DEV-12	Recuperar una versión funcional tras un despliegue defectuoso.	Demostración de recuperación y verificación posterior.
DEV-13	Reconstruir el ambiente mediante procedimiento documentado.	Instrucciones desde una suscripción autorizada, con prerrequisitos explícitos.
DEV-14	Controlar el consumo de Azure sin sobrepasar US$100.	Estimación de costos, cargos observados, alertas de presupuesto y procedimientos de detención/eliminación.

La reproducibilidad admite una inicialización mínima documentada para establecer identidades y el backend de estado. No se exige operar múltiples clústeres, alta disponibilidad entre zonas ni tolerancia a la pérdida completa de la máquina virtual que aloja K3s. Las intervenciones manuales imprescindibles se identifican y se justifican.

3.3. Métricas de Software y observabilidad

ID	Requisito	Evidencia mínima
MET-01	Definir indicadores técnicos y de comportamiento del sistema pertinentes al escenario.	Catálogo con definición, unidad, fuente, forma de cálculo y propósito.
MET-02	Incorporar logs estructurados para operaciones y fallas relevantes.	Registros con identificadores que permitan correlación.
MET-03	Instrumentar métricas de aplicación y relacionarlas con métricas de infraestructura.	Consultas y series temporales verificables.
MET-04	Instrumentar trazas con OpenTelemetry para operaciones relevantes que atraviesen componentes.	Trazas consultables y correlación razonable entre señales.
MET-05	Construir dashboards que respondan preguntas operativas concretas.	Paneles interpretables, con unidades y ventanas temporales.
MET-06	Configurar alertas vinculadas a condiciones justificadas.	Regla, evidencia de activación y explicación de posibles falsos positivos.
MET-07	Analizar al menos una degradación inducida y una recuperación.	Secuencia temporal, evidencia, interpretación y limitaciones.

Instalar una herramienta de monitoreo no satisface estos requisitos. Los indicadores, señales y análisis exigidos se detallan en el Anexo II (sección 8.2); si existe diferencia de redacción, prevalece ese anexo.

4. Kubernetes

Toda aplicación del proyecto se ejecuta en un clúster K3s instalado en máquinas virtuales de Azure, y el equipo debe demostrar que comprende su operación, no solo que lo instaló. Esta sección concreta los requisitos DEV-09 a DEV-12 y las condiciones 5.4 y 5.5 de la sección 5. Los servicios de persistencia, mensajería y almacenamiento pueden ser administrados por Azure o desplegarse en el clúster, siempre que los costos y el riesgo estén justificados.

4.1. Qué se espera del clúster

Capacidad	Qué se espera	Evidencia en la defensa
Instalación	K3s se instala de forma automatizada (Terraform y configuración versionada), sin pasos manuales no declarados.	Procedimiento de reconstrucción y nodos en estado `Ready`.
Workloads	Los componentes se despliegan con Deployments u objetos equivalentes, desde imágenes versionadas y trazables al commit.	Relación entre la imagen en ejecución, el workflow y el commit.
Configuración y secretos	La configuración y los secretos viven fuera de la imagen (ConfigMaps, Secrets u otro mecanismo justificado). Los secretos no se versionan.	Manifiestos sin secretos y explicación de cómo llegan al clúster.
Red	Los componentes se comunican mediante Services; la exposición pública es la mínima necesaria y usa transporte cifrado.	Servicios y reglas de entrada, y puertos abiertos en Azure.
Persistencia	Los datos que deben sobrevivir se almacenan fuera del ciclo de vida del Pod.	Reinicio de Pod sin pérdida de datos (COM-03).
Salud	Cada workload define probes según su comportamiento: readiness para recibir tráfico, liveness para reiniciar, startup si el arranque es lento.	Interpretación de una probe fallida y su efecto.
Recursos	Requests y límites justificados con consumo observado.	Relación entre requests, límites y métricas de uso.
Ambientes	Validación y producción se separan, por ejemplo con namespaces, con configuración y datos de prueba distintos.	Cómo se determina qué versión opera en cada ambiente.
Actualización y rollback	Una actualización defectuosa se revierte a la versión funcional previa (DEV-12).	Rollback demostrado y verificación posterior.
Acceso y permisos	El acceso administrativo está restringido y los pipelines tienen permisos mínimos documentados.	Roles, cuentas de servicio y reglas de red.
Diagnóstico	El equipo diagnostica fallas con estado, eventos y logs de los objetos.	Resolución guiada de una falla provocada por el docente.

4.2. Dimensionamiento y límites del alcance

La documentación de K3s establece como mínimo 2 núcleos y 2 GB de memoria para un nodo servidor (requisitos de K3s). Ese mínimo no contempla que aplicación, persistencia y herramientas de

observabilidad compartan el nodo, por lo que el dimensionamiento de la máquina virtual se justifica con los requests definidos y los consumos observados (DEV-11 y DEV-14).

No se exige operar múltiples clústeres, alta disponibilidad entre zonas, dos nodos ni dos réplicas en todos los escenarios. El Escenario 5 sí ejecuta dos réplicas del componente colaborativo durante su prueba específica, aunque estén en un nodo. Un reinicio de Pod no equivale a la pérdida de toda la máquina virtual, y el rollback de un Deployment no restaura datos.

Sobre los Secrets

Un Secret de Kubernetes no es, por sí solo, una solución de cifrado. El equipo explica qué protege su mecanismo de secretos, qué no protege y qué permisos permiten leerlo.

4.3. Preguntas que se pueden plantear en la defensa

- Interpretar un manifiesto: qué ocurre con el tráfico si la readiness probe falla en una réplica.
- Predecir el comportamiento de un Pod cuyo límite de memoria es menor que su consumo real.
- Identificar, a partir de eventos y logs, por qué un Pod no inicia (imagen, configuración, recursos o probes).
- Detener un Pod de forma controlada y explicar qué se recupera y qué no.
- Revertir a la versión anterior y demostrar cómo se verifica que la versión activa es la esperada.

5. Condiciones generales de aceptación

Estas condiciones se aplican a todos los escenarios; las precisiones de cada escenario las concretan y prevalecen en caso de diferencia.

5.1. Definiciones comunes. Una operación *aceptada* es aquella para la que el sistema emite una respuesta de aceptación con identificador verificable después de registrar de forma durable la información mínima necesaria para cumplirla o recuperarla. Una respuesta de aceptación no significa procesamiento completado. Una operación *confirmada* es aquella cuyo efecto ya fue aplicado, guardado de forma persistente y reconocido ante quien lo solicitó. Se considera *pérdida* la desaparición de una operación aceptada o confirmada, o de su resultado, después de reinicios de componentes, salvo eliminación explícita autorizada. Los registros rechazados por validación no cuentan como operaciones aceptadas.

5.2. Consistencia, reintentos y tiempos. Los equipos explican si cada operación se completa de forma síncrona o asíncrona, qué hace el sistema cuando desconoce el resultado de una solicitud, cómo identifica duplicados y cuál es el alcance temporal de sus garantías. No se exige semántica general de exactamente una ejecución. Los resultados comprometidos por los requisitos se verifican con datos de prueba, y los efectos duplicados que el diseño admita se identifican, se documentan y son observables.

5.3. Seguridad y datos. Las aplicaciones usan únicamente cuentas y datos ficticios. Todo componente expuesto públicamente exige los controles de autenticación que correspondan al escenario y usa transporte cifrado para credenciales y datos sensibles; se documenta el origen del nombre DNS y del certificado. No se permiten contraseñas, claves, tokens ni kubeconfigs en el repositorio (aunque sea privado), en las imágenes ni en los artefactos de construcción. El acceso de

administración de K3s está restringido y no se expone sin protección la API de Kubernetes (puerto 6443 por defecto en K3s). Se documentan los permisos de Azure y Kubernetes, incluidos los de pipelines.

5.4. Ambientes. Se exigen ambientes lógicos de desarrollo y producción, con configuraciones y datos de prueba separados; no se exigen dos clústeres ni dos máquinas virtuales. El equipo explica cómo se previenen promociones accidentales y cómo se determina qué versión opera en cada ambiente. Pueden implementarse con namespaces u otra separación verificable compatible con K3s.

5.5. Kubernetes y datos. Las aplicaciones se despliegan en K3s sobre máquinas virtuales de Azure (sección 4). El Escenario 5 ejecuta dos réplicas del componente colaborativo durante su prueba específica.

5.6. Terraform y arranque inicial. Se versionan las definiciones necesarias para crear los recursos de Azure y preparar de forma automatizada el entorno de K3s. Se admite un procedimiento inicial documentado para identidad, credenciales de federación, backend remoto y permisos indispensables; las demás acciones manuales se declaran y no sustituyen el aprovisionamiento normal. El estado remoto controla accesos, evita escrituras concurrentes incorrectas y no se publica.

5.7. Entrega y rollback. Se demuestra un pipeline de PR que valida código e IaC, otro de publicación versionada y un proceso de despliegue controlado. Una actualización defectuosa puede revertirse a la versión funcional previa; si existen migraciones de datos, son compatibles o tienen un procedimiento seguro de recuperación. No se exige restauración de datos por el simple rollback de un Deployment.

5.8. Pruebas reproducibles. Cada equipo entrega un comando o procedimiento repetible para crear datos ficticios, ejecutar las pruebas funcionales, generar carga y provocar al menos una falla controlada, con resultado esperado y una comprobación objetiva, no solo capturas de pantalla.

5.9. Costos y disponibilidad. El consumo agregado no supera US$100 de la suscripción asignada. Se documentan cargos observados, límites de capacidad, medidas para reducir gasto y la forma de detener o destruir recursos. No se exige servicio 24/7, pero durante la defensa el sistema está disponible o puede levantarse con el procedimiento documentado. Las alertas de presupuesto de Azure notifican el gasto, pero no detienen recursos por sí mismas, y una máquina virtual desasignada deja de generar cargo de cómputo mientras que sus discos y direcciones IP públicas reservadas pueden seguir generando cargo.

5.10. Alcance de la interfaz. Los Escenarios 1 y 3 pueden demostrarse mediante API y scripts; los Escenarios 2 y 4 pueden usar una interfaz sencilla o API; el Escenario 5 exige un frontend web mínimo para interacción simultánea. La estética no tiene puntuación específica.

5.11. Evidencia mínima del incidente. Se genera al menos una degradación controlada y reversible. La evidencia contiene línea de tiempo, síntoma, señal de detección, hipótesis, logs, métricas o trazas pertinentes, mecanismo causal, respuesta operativa y validación de la recuperación. Un dashboard sin análisis no satisface esta condición.

5.12. Pruebas durante la defensa. El docente puede modificar datos de entrada, pedir la interpretación de una salida, detener un Pod de forma controlada o simular una dependencia fallida. Se usan cambios reversibles, sin operaciones que consuman crédito innecesariamente ni destruyan datos del proyecto.

5.13. Pruebas comunes. Además de las pruebas de cada escenario, el equipo debe poder ejecutar pruebas y validaciones automáticas; identificar una versión desplegada y relacionarla con su código; mostrar que un cambio de infraestructura genera un plan revisable; recuperar un despliegue defectuoso sin afirmar que el rollback restaura los datos; inducir y diagnosticar un fallo controlado con logs, métricas y trazas; y documentar cómo se detiene o elimina la infraestructura. Los umbrales de cada escenario son requisitos académicos obligatorios, comprobables en las condiciones allí especificadas, y no representan objetivos empresariales de capacidad a gran escala.

6. Escenarios empresariales

Cada equipo implementa un único escenario. Las precisiones de comportamiento y las pruebas de cada uno son obligatorias y se comprueban con datos y herramientas reproducibles preparados por el equipo.

6.1. Escenario 1: Notificaciones multicanal

Una empresa de software mantiene distintas aplicaciones que deben notificar eventos a sus usuarios. Cada aplicación envía mensajes de forma independiente y los errores de los proveedores externos son difíciles de investigar. La empresa solicita una plataforma compartida que reciba solicitudes de notificación, administre su ejecución y permita consultar el resultado de cada entrega.

ID	Requisito
ESC1-01	Registrar al menos dos aplicaciones cliente e identificar cuál origina cada solicitud.
ESC1-02	Autenticar las aplicaciones que solicitan envíos y rechazar clientes no autorizados.
ESC1-03	Aceptar solicitudes con destinatario, canal, contenido y clave opcional de idempotencia.
ESC1-04	Ofrecer dos canales: correo de prueba y notificaciones internas consultables por destinatario.
ESC1-05	Registrar estado de cada solicitud y sus transiciones: aceptada, pendiente, en proceso, entregada o fallida.
ESC1-06	Consultar el estado e historial de intentos de cada notificación.
ESC1-07	Reintentar fallas transitorias hasta un máximo configurable; no reintentar automáticamente errores permanentes.
ESC1-08	Permitir reprocesar una notificación fallida mediante una acción explícita autorizada.
ESC1-09	Evitar la creación de nuevos trabajos de entrega cuando se repite una solicitud con la misma clave de idempotencia dentro de su alcance definido.
ESC1-10	Consultar recuentos y tasas de entregas por canal y estado.

Precisiones de comportamiento. La clave de idempotencia se interpreta por aplicación cliente, canal y clave durante al menos 24 horas desde la primera aceptación (la verificación puede realizarse con una ventana configurable más corta, siempre que el valor por defecto cumpla las 24 horas). Retransmitir datos idénticos devuelve la referencia original; repetir la clave con contenido diferente se rechaza. *Entregada* para correo de prueba significa aceptación comprobable por el proveedor o simulador, no lectura del destinatario; para notificación interna significa que quedó disponible de forma persistente para consulta del destinatario. Los errores transitorios reproducibles (por ejemplo HTTP 503 o timeout) generan reintentos con un máximo de tres intentos totales de entrega por notificación (el intento inicial más dos reintentos), configurable y con una espera entre intentos documentada; los permanentes (por ejemplo destinatario inválido) terminan como fallidos. Cada

intento tiene resultado y marca temporal. Una solicitud es válida cuando cumple el formato del contrato de la API, con independencia de su resultado de entrega. Los 100 envíos incluyen ambos canales y conservan los 100 registros, aunque ciertos envíos deliberadamente fallidos terminen como fallidos. Una respuesta HTTP exitosa no equivale a una entrega exitosa.

Prueba	Descripción
P1-1 Carga controlada	Enviar un lote de 100 solicitudes válidas distribuidas entre canales y verificar que todas tienen identificador y estado consultable.
P1-2 Dependencia degradada	Simular respuestas temporales de error del proveedor de correo: las solicitudes aceptadas permanecen registradas y se ejecutan los reintentos definidos, mientras el canal interno sigue funcionando.
P1-3 Idempotencia	Retransmitir cinco veces una misma solicitud con la misma clave, en una prueba secuencial y en otra concurrente, y comprobar que existe un único trabajo de entrega y que todas las respuestas remiten a la misma notificación.
P1-4 Recuperación	Reiniciar el componente que procesa entregas mientras existan trabajos pendientes y demostrar que los estados no desaparecen.
P1-5 Evidencia	Correlacionar una solicitud de extremo a extremo y justificar una alerta por aumento de fallas o retraso de entrega.

No se exige garantizar exactamente una entrega externa ante fallas ambiguas del proveedor; el equipo reconoce ese límite. Se permite un proveedor de correo de pruebas o un simulador reproducible; quedan excluidos SMS de pago, campañas comerciales y correo masivo real.

6.2. Escenario 2: Procesamiento documental

Una empresa administrativa recibe documentos digitales de clientes y necesita validarlos, almacenarlos y generar resultados consultables. Algunos procesamientos tardan más que una solicitud HTTP normal. La empresa requiere recibir nuevos documentos mientras se procesan trabajos anteriores y evitar la pérdida de solicitudes ante interrupciones.

ID	Requisito
ESC2-01	Registrar usuarios y asociarlos a una de al menos dos organizaciones de prueba.
ESC2-02	Cargar archivos PDF, PNG o JPEG de hasta 10 MB; rechazar formatos y tamaños no admitidos, validando el tipo por el contenido del archivo y no solo por su extensión.
ESC2-03	Asignar identificador, propietario, fecha, tipo y estado a cada documento.
ESC2-04	Conservar el archivo original y hacerlo accesible únicamente a usuarios autorizados de su organización.
ESC2-05	Extraer metadatos verificables, como tipo, tamaño, número de páginas cuando corresponda y dimensiones de imágenes.
ESC2-06	Generar un resultado real de procesamiento: miniatura de imagen o vista previa de la primera página de PDF.
ESC2-07	Registrar y consultar estados pendiente, en ejecución, completado y fallido.
ESC2-08	Consultar o descargar resultados completos y revisar intentos de procesamiento.
ESC2-09	Reintentar trabajos fallidos sin generar resultados repetidos indebidamente.
ESC2-10	Impedir la lectura cruzada de documentos entre organizaciones.

Precisiones de comportamiento. Un documento se considera aceptado solamente cuando se conservan de forma durable sus metadatos esenciales y el archivo original, o cuando existe un procedimiento de reconciliación documentado y comprobado que garantiza el registro consistente de ambos. La extracción y transformación se realizan realmente, sin resultados fijos precalculados. El lote de 30 archivos incluye PDF, PNG y JPEG, al menos un archivo no válido y dos organizaciones; los inválidos se registran como rechazos y no cuentan dentro de los 30 válidos. Para verificar concurrencia se mantienen al menos cinco trabajos pendientes o activos y se demuestra que se aceptan nuevas cargas. Cuando se reinicia un procesador, un trabajo aceptado puede reintentarse o finalizar en estado fallido recuperable, pero no desaparece ni destruye su original. Los resultados se identifican por documento y operación; un reintento no genera múltiples resultados activos indistinguibles.

Prueba	Descripción
P2-1 Lote de prueba	Cargar 30 documentos válidos de tamaños y formatos variados, más al menos un archivo inválido, mientras hay trabajos en proceso; verificar que los trabajos aceptados concluyen o quedan con error registrado.
P2-2 Interrupción	Finalizar una instancia de procesamiento mientras tiene trabajo asignado y demostrar recuperación o reintento sin pérdida del archivo original.
P2-3 Concurrencia	Mantener nuevas cargas disponibles mientras al menos cinco trabajos están pendientes o ejecutándose.
P2-4 Autorización	Intentar descargar un archivo de otra organización y comprobar el rechazo.
P2-5 Evidencia	Distinguir mediante telemetría el tiempo de espera del tiempo efectivo de procesamiento e identificar el documento asociado a un fallo.

No se requiere reconocimiento óptico de caracteres, firma digital, clasificación automática mediante IA ni procesamiento de videos. La interfaz de carga y consulta es opcional si la demostración mediante API es completa y verificable.

6.3. Escenario 3: Ingesta y análisis de eventos

Una organización de mantenimiento recibe eventos de dispositivos que informan temperatura, humedad y estado de operación. Los reportes pueden duplicarse, llegar fuera de orden y concentrarse en ráfagas. La organización necesita conocer el estado de sus instalaciones, consultar históricos y detectar lecturas fuera de límites establecidos.

ID	Requisito
ESC3-01	Registrar dispositivos asociados a una instalación y un identificador estable.
ESC3-02	Recibir eventos que incluyan identificador de evento (único por dispositivo), dispositivo, marca temporal y valores de temperatura, humedad y estado.
ESC3-03	Validar el formato y los rangos declarados para cada campo, registrando rechazos.
ESC3-04	Conservar eventos válidos confirmados y permitir consulta por dispositivo e intervalo de tiempo.
ESC3-05	Evitar que retransmisiones del mismo identificador distorsionen recuentos y agregados.
ESC3-06	Calcular mínimos, máximos y promedios por dispositivo en períodos seleccionados.
ESC3-07	Registrar alertas de negocio cuando una lectura supere límites configurables.

ID	Requisito
ESC3-08	Mantener correctamente la secuencia temporal de consultas aunque los eventos lleguen fuera de orden.
ESC3-09	Mostrar cantidad de eventos recibidos, validados, procesados, rechazados y pendientes, con cifras consistentes entre sí y con la relación entre ellas (incluido el tratamiento de duplicados) documentada.
ESC3-10	Proporcionar un generador o simulador reproducible de eventos, con carga y errores configurables.

Precisiones de comportamiento. Cada evento incluye identificador único por dispositivo, instante de generación en UTC y datos medidos. Se rechazan eventos con campos obligatorios ausentes, rangos inválidos o dispositivo inexistente, y no cuentan dentro de los 1,000 válidos. Una respuesta de recepción exitosa exige persistencia durable directa o en un mecanismo de recepción recuperable. Se incluyen al menos 50 duplicados y 50 eventos fuera de orden en una prueba adicional identificada. Los agregados se calculan por marcas temporales de generación, no por orden de llegada, y se define explícitamente el intervalo consultado (inicio inclusivo y fin exclusivo). En carga sostenida se envían 2,400 eventos únicos a 20 eventos/s durante 120 segundos; se registran aceptación, rechazo, pérdida, latencia y trabajo pendiente, sin exigir un tiempo fijo de finalización del procesamiento. Tras recuperar un procesador detenido, el backlog se reduce hasta cero en una ejecución de prueba sin nueva entrada.

Prueba	Descripción
P3-1 Ingesta	Aceptar una ráfaga reproducible de 1,000 eventos válidos, con concurrencia y duración definidas en el generador, sin pérdida de los eventos confirmados.
P3-2 Flujo sostenido	Ejecutar una carga de referencia de 20 eventos por segundo durante dos minutos y registrar el comportamiento observado, incluidos errores y retrasos.
P3-3 Duplicados y desorden	Retransmitir eventos anteriores y alterar el orden de llegada; verificar que consultas y agregados permanecen correctos.
P3-4 Interrupción	Detener un componente de procesamiento cuando existan eventos pendientes y comprobar su recuperación.
P3-5 Evidencia	Cuantificar tasa de entrada, tasa de procesamiento y retraso, e investigar una degradación provocada por reducción temporal de capacidad.

No se exigen dispositivos físicos, mapas, análisis predictivo ni servicios específicos de IoT. La carga se evalúa con la herramienta reproducible preparada por el equipo y las restricciones comunes del proyecto.

6.4. Escenario 4: Búsqueda e indexación

Una organización administra un catálogo de documentos y registros de conocimiento que se actualiza de forma continua. Los usuarios necesitan búsquedas por texto, filtros y ordenación de resultados. Cuando se incorpora o actualiza un registro, puede existir un intervalo antes de que el cambio sea visible en las búsquedas. La organización debe conocer ese retraso y mantener las consultas disponibles durante tareas de mantenimiento e indexación.

ID	Requisito
ESC4-01	Registrar documentos de catálogo con identificador, título, descripción, etiquetas, categoría, fecha y estado de publicación.
ESC4-02	Crear, modificar y despublicar registros de manera autorizada.
ESC4-03	Buscar por texto en título y descripción, combinando términos con filtros por etiqueta y categoría.
ESC4-04	Ordenar y paginar resultados, devolviendo el total de coincidencias.
ESC4-05	Garantizar que los registros no publicados no aparezcan en búsquedas ordinarias.
ESC4-06	Incorporar cambios del catálogo al mecanismo de búsqueda e informar si un registro está pendiente de actualización.
ESC4-07	Disponer de una operación administrativa autorizada para reconstruir la estructura de búsqueda desde el catálogo persistente.
ESC4-08	Conservar los datos fuente incluso si falla o se reconstruye el mecanismo de búsqueda.
ESC4-09	Permitir consultar el estado de actualización del catálogo y los errores ocurridos.
ESC4-10	Proporcionar un conjunto reproducible de registros de prueba generado por el equipo, sin datos privados reales.

Precisiones de comportamiento. El *catálogo fuente* es el registro persistente autoritativo. La solución funciona con 2,000 entradas ficticias deterministas y al menos 100 modificaciones o despublicaciones adicionales. Una escritura confirmada permanece en el catálogo aunque temporalmente no aparezca en la búsqueda. El sistema ofrece una manera consultable de distinguir registro guardado de registro actualizado para búsqueda; una vez que se reporta la actualización como completada, las búsquedas ordinarias reflejan alta, edición o despublicación. Los filtros pueden combinarse y la paginación tiene orden determinista para evitar páginas ambiguas. La detención del proceso de actualización no borra el catálogo fuente; tras restaurarlo o ejecutar una reconstrucción autorizada, se compara un conjunto conocido de consultas con resultados esperados. No se exige un tiempo máximo fijo de indexación, pero sí medirlo y justificar su comportamiento. El requisito ESC4-05 se evalúa una vez que la actualización se reporta como completada; mientras exista una actualización pendiente, el sistema lo informa (ESC4-06) y el equipo documenta el comportamiento de las búsquedas en esa ventana, incluido el riesgo de mostrar temporalmente un registro ya despublicado.

Prueba	Descripción
P4-1 Datos	Cargar 2,000 registros de prueba reproducibles y ejecutar búsquedas con texto, filtros, ordenación y paginación, contrastando con resultados esperados.
P4-2 Actualización	Modificar y despublicar registros y medir cuánto tardan en reflejarse en las consultas; el resultado no contiene registros despublicados una vez confirmada su actualización.
P4-3 Interrupción	Provocar la indisponibilidad temporal del proceso que actualiza el mecanismo de búsqueda; los cambios pendientes se conservan o pueden reconstruirse desde el catálogo fuente.
P4-4 Recuperación	Reconstruir el mecanismo de búsqueda y comparar resultados con el catálogo persistente.
P4-5 Evidencia	Medir latencia de búsqueda, errores y retraso de actualización; explicar el comportamiento durante la interrupción.

No se exige implementar desde cero un motor de búsqueda: se admiten capacidades de bases de datos o herramientas existentes, siempre que la solución cumpla los requisitos. La interfaz de búsqueda es recomendable, pero opcional si la demostración mediante API es verificable. No se requiere búsqueda semántica, embeddings ni IA generativa.

6.5. Escenario 5: Colaboración en tiempo real

Una organización necesita tableros compartidos para coordinar tareas de trabajo. Varios usuarios pueden modificar un mismo tablero desde distintas sesiones y deben observar los cambios sin recargar manualmente la página. La plataforma debe conservar los cambios confirmados y restablecer el estado después de desconexiones o reinicios de componentes.

ID	Requisito
ESC5-01	Registrar usuarios y crear tableros de trabajo con miembros autorizados.
ESC5-02	Crear tareas con título, descripción breve, responsable y estado.
ESC5-03	Modificar y mover tareas entre estados pendiente, en progreso y completada.
ESC5-04	Propagar cambios confirmados a otros usuarios conectados al mismo tablero sin recarga manual.
ESC5-05	Conservar los cambios confirmados en almacenamiento persistente.
ESC5-06	Impedir que usuarios ajenos al tablero consulten o modifiquen sus tareas.
ESC5-07	Resolver modificaciones concurrentes mediante una política explícita y observable; no permitir pérdidas silenciosas.
ESC5-08	Reconectar clientes después de una desconexión y sincronizar su estado con la versión vigente del tablero.
ESC5-09	Registrar un historial básico de modificaciones con autor, fecha y operación.
ESC5-10	Proporcionar una interfaz web mínima que permita la interacción simultánea de varios usuarios.

Precisiones de comportamiento. Una modificación se considera confirmada cuando ha sido persistida y el cliente recibe confirmación. El tablero compartido muestra una versión o mecanismo equivalente que permite detectar conflictos; ante cambios concurrentes sobre la misma tarea no se admite sobrescritura silenciosa de cambios confirmados. Una solución válida es rechazar explícitamente el cambio basado en una versión obsoleta y solicitar recarga, sin exigir algoritmos avanzados de edición concurrente. En la prueba se conectan tres sesiones en al menos dos navegadores o perfiles aislados y se comprueba una secuencia de diez cambios confirmados sin recargas manuales. En la prueba de dos réplicas se verifica, mediante evidencia de Kubernetes, que las sesiones se atienden efectivamente por réplicas distintas. La reconexión puede producir una interrupción temporal de sesión, pero debe recuperar desde almacenamiento persistente el estado confirmado, sin perder cambios.

Prueba	Descripción
P5-1 Sincronización	Conectar tres sesiones de usuario y comprobar la propagación automática de creaciones, modificaciones y cambios de estado.
P5-2 Concurrencia	Generar dos modificaciones conflictivas sobre una misma tarea y comprobar la política definida, mostrando al usuario el resultado o rechazo correspondiente.
P5-3 Réplicas	Operar al menos dos instancias del componente que mantiene conexiones y verificar la propagación de cambios entre clientes atendidos por instancias distintas.

Prueba	Descripción
P5-4 Interrupción	Reiniciar una instancia durante la demostración; los clientes afectados pueden reconectarse y recuperar el estado confirmado.
P5-5 Evidencia	Medir latencia de propagación, conexiones activas y reconexiones; correlacionar una modificación con su recepción por otro cliente.

Se excluye la edición colaborativa de texto carácter por carácter, la videoconferencia, el chat multimedia y la colaboración offline prolongada. No se exige que una conexión sobreviva a la terminación de su instancia: se evalúa la reconexión y la consistencia del estado confirmado.

7. Evaluación y defensa

Cada rúbrica usa 100 puntos propios y su conversión a la nota de cada asignatura es la siguiente:

Asignatura	Solución grupal	Defensa o comprobación individual	Peso total en la nota final
DevOps	Calificación sobre 100 × 0.15	Calificación sobre 100 × 0.15	30%
Métricas de Software	Calificación sobre 100 × 0.10	Calificación sobre 100 × 0.10	20%

La defensa de DevOps y Métricas son evaluaciones diferenciadas, aunque puedan celebrarse consecutivamente con la misma infraestructura; no se concede automáticamente la misma calificación en ambas. La defensa individual puede abordar cualquier componente del sistema, aunque lo haya desarrollado principalmente otro integrante: interpretación de un plan de Terraform, diagnóstico de un workload, trazabilidad, recuperación de una versión anterior o explicación de un fallo en CI/CD.

7.1. Anexo I. Evaluación de DevOps

Rúbrica grupal (100 puntos)

Criterio	Puntos	Evidencia y condición de logro
Funcionalidad y arquitectura	15	Requisitos del escenario, pruebas reproducibles, diagrama implementado y decisiones justificadas.
Contenedores e integración continua	15	Imágenes seguras y versionadas, pruebas en PR, protección ante fallos y trazabilidad.
Infraestructura con Terraform	20	Recursos Azure versionados, estado remoto protegido, plan revisable y aplicación controlada; instalación automatizada de K3s.
Kubernetes y operación	20	K3s operativo, configuración, secretos, red, persistencia, salud, requests/límites y diagnóstico.
Entrega continua y recuperación	20	Promoción controlada, versión identificable, rollback demostrado y tratamiento apropiado de cambios de datos.
Reproducibilidad, seguridad y costos	10	Reconstrucción documentada, mínimo privilegio, ausencia de secretos expuestos y evidencia del consumo máximo.
Total	**100**	

Escala por criterio: 100% si está implementado y demostrado de forma correcta y reproducible; 75% si funciona con defectos menores documentados que no invalidan la capacidad; 50% si funciona parcialmente con intervención manual relevante o evidencia incompleta; 25% si hay intentos

verificables sin funcionamiento demostrable; 0% si no existe evidencia. El docente fundamenta cualquier puntuación intermedia en evidencias específicas.

Rúbrica de defensa individual (100 puntos)

Criterio	Puntos	Comprobación
Comprensión de la arquitectura y compromisos	25	Explica componentes, flujos, decisiones y alternativas.
Diagnóstico técnico	30	Formula hipótesis, examina evidencia y determina el mecanismo probable de falla.
Operación y recuperación	25	Ejecuta o describe de forma verificable cambios, rollback y verificaciones posteriores.
IaC, seguridad y trazabilidad	20	Interpreta workflows, planes de Terraform, identidades, permisos y versiones.
Total	**100**	

Escala individual: 100% para demostración autónoma, técnicamente correcta y justificada; 75% para ejecución o explicación correcta con ayudas menores del docente; 50% para comprensión parcial con errores significativos; 25% para respuestas fragmentarias sin autonomía; 0% sin evidencia de dominio. No se permite IA durante esta evaluación.

7.2. Anexo II. Evaluación de Métricas de Software

El proyecto integrador de Métricas de Software pesa 20% de la nota final: 10% de solución grupal y 10% de comprobación individual, cada uno con su rúbrica independiente de 100 puntos. La aplicación y la infraestructura son compartidas con DevOps, pero aquí no se califica de nuevo su implementación, sino la medición, la instrumentación y el razonamiento basado en evidencia.

Requisitos mínimos de medición

1. Formular al menos tres preguntas operativas diferenciadas y justificar qué decisiones permite tomar cada una.
2. Definir tres SLIs (al menos uno específico del comportamiento distintivo del escenario), con población de eventos, numerador y denominador o unidad, ventana de cálculo, origen de datos y exclusiones; establecer dos SLOs numéricos correspondientes, con ventana de evaluación, justificación y presupuesto de error calculable cuando aplique. Pueden usarse ventanas de prueba; no se exige demostrar un SLO mensual real.
3. Implementar logging estructurado de errores y transiciones significativas, con timestamps, severidad, componente, resultado e identificador de operación o correlación; evitar datos sensibles en los registros.
4. Instrumentar al menos cuatro métricas de aplicación (incluida al menos una de resultado y una de latencia o espera), además de métricas relevantes de CPU o memoria cuando sustenten el análisis; documentar unidades, etiquetas, frecuencia de recolección y riesgos de cardinalidad.
5. Mostrar al menos una operación relevante mediante OpenTelemetry, con spans de dos componentes o etapas distinguibles, propagación de contexto y explicación de tiempos; las operaciones asíncronas conservan una relación trazable mediante propagación o enlaces entre spans.

6. Entregar un dashboard funcional con al menos seis paneles útiles que cubran resultados del servicio, latencia o espera, errores y capacidad, con unidades y períodos de observación identificables.
7. Configurar al menos dos reglas de alerta: una por degradación del servicio o de un SLI y otra por un síntoma operativo pertinente. Demostrar el disparo de al menos una y su retorno al estado normal, y justificar umbral, ventana y posibles falsos positivos.
8. Realizar un experimento de degradación controlada, con hipótesis inicial, procedimiento reproducible, señales recogidas, análisis de causa, mitigación, verificación posterior, limitaciones y al menos una consulta o evidencia comparativa antes y después.
9. Versionar la configuración y explicar cómo se restablecen instrumentación, recolección, paneles y alertas sin depender de configuraciones manuales no documentadas.

No se impone una única pila. Las trazas usan OpenTelemetry; Prometheus y Grafana son las herramientas de referencia del curso y se admiten alternativas funcionalmente equivalentes que permitan demostrar las consultas, alertas y análisis exigidos.

Rúbrica grupal (100 puntos)

Criterio	Puntos	Evidencia exigida
Objetivos, SLIs, SLOs y presupuestos de error	20	Preguntas, definiciones operativas, cálculo, ventanas, límites y justificación.
Logging estructurado y correlación	15	Registros significativos, contexto preservado y consultas de investigación.
Métricas e interpretación	20	Instrumentación válida, unidades, consultas, tasas y latencias, y análisis de su significado.
Trazas distribuidas con OpenTelemetry	15	Propagación, spans pertinentes y análisis de una operación.
Dashboards y alertas	15	Paneles orientados a preguntas, condiciones justificadas y alerta probada.
Investigación de incidente y reproducibilidad	15	Hipótesis, evidencia correlacionada, antes y después, limitaciones, configuración versionada
Total	**100**	

Escala grupal: 100% cuando las evidencias son correctas, reproducibles y permiten sustentar una conclusión; 75% con imprecisiones menores que no afectan la interpretación central; 50% cuando la implementación o el análisis es parcial; 25% cuando existen artefactos sin utilidad demostrada; 0% cuando falta evidencia. Métricas sin definición operacional, dashboards decorativos o alertas sin criterio técnico no constituyen cumplimiento completo.

Rúbrica de comprobación individual (100 puntos)

Criterio	Puntos	Comprobación
Razonamiento de medición, SLIs y SLOs	25	Formula e interpreta indicadores, ventanas, limitaciones y presupuesto de error.
Interpretación de logs, métricas y trazas	35	Correlaciona señales, interpreta latencia y discrimina causas probables.
Investigación de incidentes	30	Plantea hipótesis, propone comprobaciones y fundamenta la remediación con evidencia.
Instrumentación y limitaciones	10	Explica configuración, muestreo, cardinalidad, costos y límites de la observación.

Criterio	Puntos	Comprobación
Total	**100**	

Escala individual: 100% para argumentación autónoma, causal y sustentada; 75% para interpretación esencialmente correcta con omisiones menores; 50% para razonamiento parcial o evidencia insuficiente; 25% para afirmaciones vagas o lecturas erróneas; 0% sin demostración de comprensión. No se permite IA durante la comprobación.

Preguntas operativas sugeridas por escenario

Escenario	Preguntas a responder con evidencia
1. Notificaciones	¿Cuántas solicitudes aceptadas se entregan dentro del objetivo? ¿Qué canal o dependencia explica reintentos y retrasos?
2. Documentos	¿Cuánto demora cada etapa desde la aceptación hasta el resultado? ¿Qué causa los trabajos pendientes o fallidos?
3. Eventos	¿La velocidad de procesamiento sigue la de recepción? ¿Cuánto retraso y backlog produce una reducción de capacidad?
4. Búsqueda	¿Cuándo pasan los cambios confirmados a ser visibles en consultas? ¿Qué impacto tiene la actualización sobre latencia y errores de búsqueda?
5. Colaboración	¿Qué retraso existe entre confirmar un cambio y observarlo en otra sesión? ¿Qué ocurre con las reconexiones y los conflictos?